S500 Agentic Drone Build Guide

Nov 6, 2026 · @Antonio

Contents

1. Overview
2. Progress
3. Phase 0: Parts and pre-checks
4. Phase 1: Simulation on Ubuntu
5. Phase 2: Firmware choice (done)
6. Phase 3: Flashing (done)
7. Phase 4: Wiring
8. Phase 5: Base setup in QGroundControl
9. Phase 6: Lidar and obstacle avoidance
10. Phase 7: Raspberry Pi 5 on TELEM3
11. Phase 8: Ground station (QGroundControl + MAVProxy + your scripts)
12. Phase 9: Flight tests
13. Testing every script
    a. Test 1: Install tron and run its unit tests (laptop)
    b. Test 2: Start one SITL session for all script tests
    c. Test 3: goto.py in SITL
    d. Test 4: target.py in SITL
    e. Test 5: nudge.py in SITL
    f. Test 6: tron in SITL
    g. Test 7: Old scripts and tron side by side in SITL
    h. Test 8: tron on the Pi (install, unit tests, camera)
    i. Test 9: Ground checks on the real drone (props off)
    j. Test 10: Field flights, old script then tron
14. Flight-day checklist
15. Troubleshooting
16. Sources

Overview

The Holybro S500 V2 runs ArduCopter 4.7.2 on its Pixhawk 6C. An RPLidar C1 plugs straight into the Pixhawk, so obstacle avoidance always runs on the flight controller. Targets come from the ground station over the SiK radio, and from the Raspberry Pi 5 over a UART. **QGroundControl** (stable release) on Ubuntu is the ground station for setup, monitoring and flying.

Who does what

- **Pixhawk 6C + ArduCopter:** always the pilot. Runs GUIDED mode, BendyRuler path planning from the lidar, and every failsafe.
- **Raspberry Pi 5:** mission manager and camera. Sends GPS targets when far away, body-relative moves when close.
- **Ground station (Ubuntu laptop):** QGroundControl shows the drone and handles setup; MAVProxy shares the radio link between QGC and your agent scripts.
- **RC transmitter (FS-iA6B receiver, PPM):** final manual override at all times.

Tools

Job	Tool
Simulation, building, script development	ArduPilot SITL + MAVProxy
Drone setup, calibration, parameters, motor test	QGroundControl
Starting tune values from prop size and battery	ArduPilot Methodic Configurator (Phase 5.7)
Sharing the radio between QGC and scripts	MAVProxy router (Phase 8)
Optional, on Windows: lidar radar view, tuning helper, log analysis	Mission Planner

The old ground scripts connect to `udpin:0.0.0.0:14551` and tron to `udpin:0.0.0.0:14553` (full port map under Testing every script). In simulation and in the field, MAVProxy feeds those ports, so the same commands run unchanged in both.

Use the **stable** QGroundControl release, not Daily: the Daily build's firmware page had no ArduPilot option.

Order of work. Simulation first, so every command and parameter is proven before it touches the aircraft. Each phase ends with a check that must pass before the next one.

Safety rules for every bench phase

- Propellers **off** for everything until Phase 9. That includes "just a quick motor test".
- Flight battery disconnected while wiring. Power the Pixhawk from USB for configuration.
- Never power the Pi 5 from a Pixhawk port.
- One change at a time, then test. After each phase that passes, save the parameters (QGC: Parameters → Tools → Save to file) into your repo's `params/` folder and commit.

Progress

Phase	Status	Notes
0 Parts and pre-checks	Done	PX4 parameters backed up, motor mapping recorded
1 Simulation	Done	Avoidance in GUIDED works with GUID_OPTIONS = 64
2 Firmware choice	Done	Copter/stable/Pixhawk6C/arducopter.apj
3 Flashing	Done	Flashed with the uploader
4 Wiring	Done	SiK TELEM1, lidar TELEM2, Pi TELEM3, FS-iA6B on RC IN (PPM)
5 Base setup in QGroundControl	Done	DShot300; internal compass off (COMPASS_USE2 = 0)
6 Lidar and avoidance	Done	Avoidance parameters set; confirm in flight (Phase 9 items 3 and 5)
7 Raspberry Pi 5	Done	nudge.py reads the mode over TELEM3 and refuses outside GUIDED
8 Ground station	In progress	Router and QGC working; target.py not yet flown
9 Flight tests	In progress	Loiter hover flown and log checked; items 2 onward to do

Phase	Status	Notes
Script tests	To do	Tests 1–10 below

Phase 0: Parts and pre-checks

Part	Notes
Holybro S500 V2 frame, motors, ESCs	Four ESCs on DShot300, 10-inch props, 4S battery
Holybro Pixhawk 6C (not Mini)	The Mini has no TELEM3, which this plan uses for the Pi
GPS/compass module	On GPS1
Holybro SiK radio pair (915 MHz)	Air unit on TELEM1, ground unit on the laptop's USB
Slamtec RPLidar C1	On TELEM2; keep its USB adapter board for bench tests
FS-iA6B receiver (PPM) + transmitter	On RC IN; manual override
Raspberry Pi 5 + microSD (32 GB+)	On TELEM3
5V 5A BEC (4S input)	Dedicated supply for the Pi 5
JST-GH 6-pin cables, Dupont jumpers	For TELEM2 → lidar and TELEM3 → Pi

Pre-checks

- [x] ~~Read the Pixhawk 6C serial number. If its middle block is **001**, TELEM3 pins 4 and 5 are I2C; fine here, because the Pi uses only TX, RX and GND, but never connect anything to those two pins.~~
- [x] ~~Back up the PX4 parameters before flashing.~~
- [x] ~~Record the PX4 motor mapping: ESCs on AUX 1–4 with DShot600, AUX1 = Motor 3, AUX2 = Motor 2, AUX3 = Motor 4, AUX4 = Motor 1.~~
- [x] ~~Note the prop size and cell count: 10-inch, 4S for the stock kit.~~
- [x] ~~Test the lidar alone on its USB adapter board with Slamtec's RoboStudio, so you know the sensor works before wiring it in.~~

Phase 1: Simulation on Ubuntu

ArduPilot's simulator (SITL) fakes an RPLidar surrounded by virtual posts, so avoidance and your scripts can be tested with no hardware. Everything here runs on your laptop in Ubuntu, not on the Pi.

1.1 One-time shell setup. Two shortcuts used throughout this guide: `$REPO` for your repo and `apenv` to activate the ArduPilot Python environment.

```
echo 'export REPO=$HOME/Documents/2026/usyd/yr2sem2/joey-
tron/tronV2adrupilot' >> ~/.bashrc
echo "alias apenv='source ~/venv-ardupilot/bin/activate'" >> ~/.bashrc
. ~/.bashrc
```

Don't make the ArduPilot venv activate automatically in every terminal. It carries setuptools 84, and colcon (your ROS 2 Jazzy builds) needs a version below 80. Activate it with `apenv` only where you're doing ArduPilot work.

1.2 Repo layout (done). Your repo `TronV2ardrupilot` holds everything for this build, with ArduPilot as a submodule pinned to Copter-4.7.2:

```
tronV2adrupilot/
├── ardupilot/   submodule, Copter-4.7.2
├── gcs/         ground station scripts: goto.py, target.py
├── agent/       tron, the production package (tests, configs)
├── pi/          Pi scripts: nudge.py, mission manager
├── params/      .params files, one per passing phase
├── firmware/    the .apj you flashed (ignored by Git)
└── logs/        flight logs (ignored by Git)
```

Check the submodule is on the right version. Your prompt shows `APMrover2-stable` because several tags can point at one commit; what matters is that Copter-4.7.2 is among them:

```
git -C $REPO/ardupilot tag --points-at HEAD | grep Copter-4.7.2
```

If that prints nothing, pin it and rebuild:

```
cd $REPO/ardupilot
git checkout Copter-4.7.2
git submodule update --init --recursive
cd $REPO && git add ardupilot && git commit -m "Pin ArduPilot to Copter-
4.7.2"
```

1.3 Prerequisites and build (done). For reference, or if you set up another machine:

```
cd $REPO/ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh
apenv
./waf configure --board sitl
./waf copter
```

Run the install script without `-y` and answer **N** to "Make ArduPilot venv default" and to "Install ArduPilot STM32 toolchain" (you flash the official `.apj`, so you don't need the compiler). If it stalls on wxPython from extras.wxpython.org, press Ctrl+C: wxPython 4.2.1 is already installed system-wide, and the venv can see it.

1.4 Start SITL with the simulated lidar and posts. In a new terminal:

```
apenv
cd $REPO/ardupilot/ArduCopter
../Tools/autotest/sim_vehicle.py -v ArduCopter -A --serial5=sim:rplidara2 -l
51.8752066,14.6487840,54.15,0 --console --map --out=udp:127.0.0.1:14551
```

The first time ever, add `-w` to wipe the simulated parameters to defaults. Never use `-w` after that, or your settings are lost.

The simulator puts its lidar on SERIAL5. On the real drone the lidar is on SERIAL2 (Phase 4), so never load a SITL parameter file onto the drone.

1.5 MAVProxy commands. Type these at the `STABILIZE>` prompt in that same terminal, **one line at a time**, waiting for each reply. Pasting several lines at once glues them together.

```
script /tmp/post-locations.scr
param set SERIAL5_PROTOCOL 11
param set PRX1_TYPE 5
param set AVOID_ENABLE 7
param set AVOID_MARGIN 2
param set OA_TYPE 1
param fetch
```

Wait for `Received ... parameters`, then:

```
param set OA_BR_LOOKAHEAD 5
param set OA_MARGIN_MAX 2
param set OA_DB_OUTPUT 3
param set GUID_OPTIONS 64
reboot
```

The first line draws the posts on the map as blue circles. `GUID_OPTIONS` = 64 is what makes GUIDED mode use BendyRuler; without it GUIDED flies straight through obstacles (Guided mode options). Changing `OA_TYPE` or `GUID_OPTIONS` needs the reboot. `OA_DB_OUTPUT` = 3 shows detected obstacles on the map as red `[OA_DB]` labels.

1.6 Manual test. After the reboot reconnects and the console shows GPS and EKF ready:

```
mode guided
arm throttle
takeoff 10
```

Right-click the map at a point straight past a post, about 20–30 m away, and choose **Fly To**. It should bend around the post, keeping about 2 m away. To make the dodge obvious, `param set OA_MARGIN_MAX 4`, then set it back to 2.

1.7 The same test from Python. Save as `$REPO/gcs/goto.py`. It arms, takes off and flies to a target, but only once you've put the drone in GUIDED yourself, and it stops the moment you switch out of GUIDED. Targets are capped at 100 m away and 30 m high. It works in SITL (Test 3) and in the field (Test 10, Flight 3):

```
import math
import sys
import time

from pymavlink import mavutil

mav = mavutil.mavlink
MAX_ALT = 30
MAX_DIST = 100


def usage():
    sys.exit("usage: goto.py CONN LAT LON ALT")
```

```
def recv(m, types, timeout):
    end = time.time() + timeout
    while time.time() < end:
        msg = m.recv_match(type=types, blocking=True, timeout=0.5)
        if msg is None:
            continue
        if msg.get_srcSystem() != m.target_system or msg.get_srcComponent()
!= m.target_component:
            continue
        if msg.get_type() == "STATUSTEXT":
            print("FC:", msg.text)
            continue
        return msg
    return None


def heartbeat_state(hb):
    return mavutil.mode_string_v10(hb), bool(hb.base_mode &
mav.MAV_MODE_FLAG_SAFETY_ARMED)


def command(m, cmd, *params):
    p = list(params) + [0] * (7 - len(params))
    m.mav.command_long_send(m.target_system, m.target_component, cmd, 0, *p)
    end = time.time() + 5
    while time.time() < end:
        ack = recv(m, ["COMMAND_ACK", "STATUSTEXT"], end - time.time())
        if ack is not None and ack.command == cmd:
            return ack.result
    return None


def distance(lat1, lon1, lat2, lon2):
    dn = (lat2 - lat1) * 111320
    de = (lon2 - lon1) * 111320 * math.cos(math.radians(lat1))
    return math.hypot(dn, de)


def watch(m, until, timeout):
    end = time.time() + timeout
    while time.time() < end:
        msg = recv(m, ["HEARTBEAT", "GLOBAL_POSITION_INT", "STATUSTEXT"], 3)
```

```
        if msg is None:
            sys.exit("lost telemetry, stopping (drone keeps its last target;
take over with the mode switch)")
        if msg.get_type() == "HEARTBEAT":
            mode, armed = heartbeat_state(msg)
            if mode != "GUIDED" or not armed:
                sys.exit(f"stopped: pilot took over (mode {mode}, armed
{armed})")
        elif until(msg):
            return msg
    sys.exit("timed out; take over with the mode switch")


if len(sys.argv) != 5:
    usage()
conn = sys.argv[1]
lat, lon, alt = (float(v) for v in sys.argv[2:5])
if not 2 <= alt <= MAX_ALT:
    sys.exit(f"alt must be 2-{MAX_ALT} m")

m = mavutil.mavlink_connection(conn, source_system=255, source_component=190)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True, timeout=30)
    if hb is None:
        sys.exit("no heartbeat from the drone")
    if hb.autopilot == mav.MAV_AUTOPILOT_ARDUPILOTMEGA and
hb.get_srcComponent() == mav.MAV_COMP_ID_AUTOPILOT1:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

m.mav.command_long_send(m.target_system, m.target_component,
mav.MAV_CMD_SET_MESSAGE_INTERVAL, 0,
                        mav.MAVLINK_MSG_ID_GLOBAL_POSITION_INT, 200000, 0, 0,
0, 0, 0)
pos = recv(m, ["GLOBAL_POSITION_INT", "STATUSTEXT"], 10)
if pos is None:
    sys.exit("no position from the drone")
d = distance(pos.lat / 1e7, pos.lon / 1e7, lat, lon)
if d > MAX_DIST:
    sys.exit(f"target is {d:.0f} m away, limit {MAX_DIST} m, refusing")

mode, armed = heartbeat_state(hb)
```

```
print(f"mode {mode}, armed {armed}, target {d:.0f} m away")
if mode != "GUIDED":
    sys.exit("not in GUIDED, refusing (flip the GUIDED switch, or 'mode
guided' in SITL)")

if not armed:
    result = command(m, mav.MAV_CMD_COMPONENT_ARM_DISARM, 1)
    if result != mav.MAV_RESULT_ACCEPTED:
        sys.exit(f"arming refused (result {result}); read the FC messages
above")
    print("armed")
    result = command(m, mav.MAV_CMD_NAV_TAKEOFF, 0, 0, 0, 0, 0, 0, alt)
    if result != mav.MAV_RESULT_ACCEPTED:
        sys.exit(f"takeoff refused (result {result}); it will disarm by
itself in a few seconds")
    print(f"taking off to {alt:.0f} m")
    watch(m, lambda p: p.get_type() == "GLOBAL_POSITION_INT" and
p.relative_alt / 1000 >= alt * 0.9, 60)
elif pos.relative_alt / 1000 < 1:
    sys.exit("armed but on the ground; disarm and run again")

m.mav.set_position_target_global_int_send(
    0, m.target_system, m.target_component,
    mav.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    0b110111111000, int(lat * 1e7), int(lon * 1e7), alt,
    0, 0, 0, 0, 0, 0, 0, 0)


def arrived(p):
    if p.get_type() != "GLOBAL_POSITION_INT":
        return False
    d = distance(p.lat / 1e7, p.lon / 1e7, lat, lon)
    print(f"{d:6.1f} m to go, alt {p.relative_alt / 1000:5.1f} m")
    return d < 2


watch(m, arrived, 30 + 3 * d)
print("arrived")
```

With SITL from 1.4 running and the drone disarmed, type mode guided at the SITL prompt, then in a second terminal:

```
apenv
cd $REPO/gcs
python3 goto.py udpin:0.0.0.0:14551 <lat> <lon> 10
```

Pick a target behind a post; the map shows the latitude and longitude under the mouse.

Phase 1 passes when: the drone bends around posts in both 1.6 and 1.7, and `goto.py` prints "arrived". Commit `goto.py`.

Phase 2: Firmware choice (done)

Use **ArduCopter 4.7.2, stable, board folder `Pixhawk6C`**, the same version as your SITL build (Copter-4.7.2 announcement).

Folder in `Copter/stable/`	Use it?	Why
`Pixhawk6C`	**Yes**	Standard multirotor firmware; supports DShot600 on the AUX outputs
`Pixhawk6C-bdshot`	No	Adds bidirectional DShot (motor RPM feedback), which the stock S500 ESCs don't need
`Pixhawk6C-heli`, `Pixhawk6C-bdshot-heli`	No	Traditional single-rotor helicopters

Inside the folder, check `git-version.txt` says Copter 4.7.2, then download `arducopter.apj` (not `arducopter_with_bl.hex`) into `$REPO/firmware/`. The `.apj` is JSON, so you can read its header without flashing:

```
cd $REPO/firmware
python3 -c "import
json;d=json.load(open('arducopter.apj'));print(d['board_id'],
d.get('git_identity'))"
```

`board_id` must be **56**, the Pixhawk 6C's ID in ArduPilot's board definition. If the file won't parse, the download is incomplete.

Phase 3: Flashing (done)

The board was flashed with ArduPilot's uploader from Ubuntu, and QGroundControl now identifies it as `Pixhawk6C [Holybro]` running ArduCopter. The methods are kept here for future updates.

Before any flash: flight battery disconnected, props off, USB cable straight into the laptop, parameters saved.

Option A: Terminal uploader (recommended, what you used)

Close QGroundControl first; it holds the USB port. Start with the Pixhawk unplugged:

```
apenv
cd $REPO/ardupilot
python3 Tools/scripts/uploader.py --port /dev/ttyACM0
$REPO/firmware/arducopter.apj
```

Plug the board in when it says it's waiting for the bootloader. It reports board ID 56, then erases, programs and verifies.

Option B: QGroundControl stable

1. Unplug the Pixhawk and open Vehicle Setup → **Firmware**.
2. Plug it in. In the dialog choose **ArduPilot**, tick **Advanced settings**, pick **Custom firmware file...**, and select `arducopter.apj`.
3. Wait for erase, program and verify before unplugging.

Confirm and reset

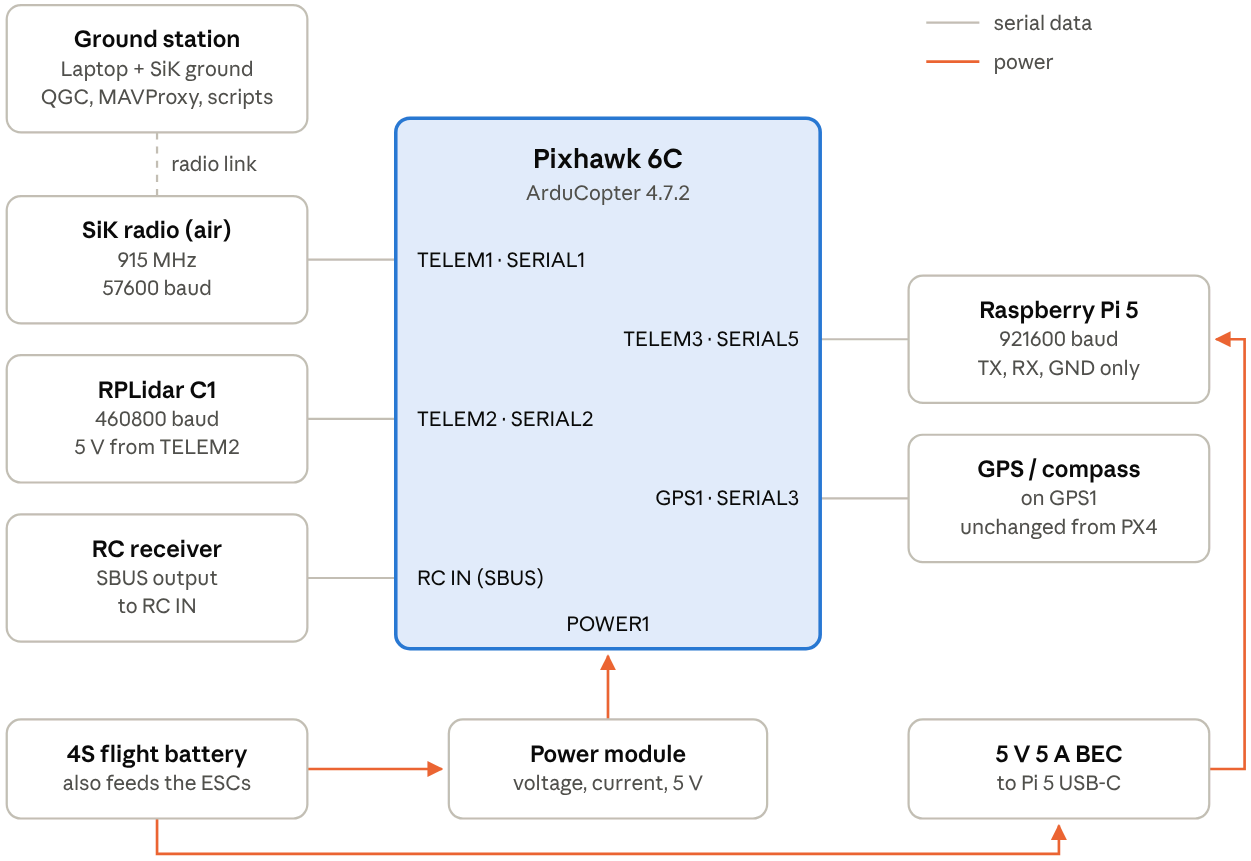
1. Open QGroundControl. It should connect as an ArduPilot vehicle, and the vehicle messages should show `ArduCopter V4.7.2` and `Pixhawk6C`.
2. Reset to defaults: **Parameters → Tools → Reset all to firmware's defaults**, then **Parameters → Tools → Reboot Vehicle**.
3. Save the parameters as `$REPO/params/s500_ardu_00_fresh.params` and commit.

If a flash fails partway, the bootloader survives. Unplug, wait 10 seconds and repeat.

Phase 4: Wiring

Every TELEM port on the 6C is a 6-pin JST-GH with 3.3 V logic, and so are the Pi 5's GPIO and the C1, so no level shifters are needed (Holybro 6C port pinouts). **Wire with the flight battery disconnected and the Pixhawk unpowered.**

Three serial devices on three TELEM ports; the Pi has its own supply



S500 wiring · 6 devices, 3 power paths

Only the lidar draws power from the Pixhawk. The Pi runs from its own 5 A supply and shares just ground with the flight controller.

Port plan

6C port	ArduPilot serial	Device	Protocol	Baud
TELEM1	SERIAL1	SiK radio (air unit)	`SERIAL1_PROTOCOL` = 2 (MAVLink2)	`SERIAL1_BAUD` = 57

6C port	ArduPilot serial	Device	Protocol	Baud
TELEM2	SERIAL2	RPLidar C1	`SERIAL2_PROTOCOL` = 11 (Lidar360)	`SERIAL2_BAUD` = 460 (460800)
TELEM3	SERIAL5	Raspberry Pi 5	`SERIAL5_PROTOCOL` = 2 (MAVLink2)	`SERIAL5_BAUD` = 921
GPS1	SERIAL3	GPS/compass	default	default
RC IN	none	FS-iA6B (PPM) receiver	detected automatically	

The SERIAL numbers don't follow the TELEM numbers: **TELEM3 is SERIAL5** on this board (ArduPilot Pixhawk 6C page). Getting this wrong is the most common reason a device "doesn't work".

TELEM pinout (all three ports)

Pin	Signal	Use here
1 (red)	+5 V out	Lidar power only, never the Pi
2	TX (out)	→ device RX
3	RX (in)	← device TX
4	CTS	Not connected
5	RTS	Not connected
6	GND	Always connected

SiK radio → TELEM1

Plug the Holybro radio's 6-pin cable straight in. TELEM1 has its own 1.5 A power limit, which covers the radio.

RPLidar C1 → TELEM2

TELEM2 pin	Signal	C1 signal
1	+5 V	VCC
2	TX	RX
3	RX	TX
6	GND	GND

- The C1 has a 5-pin XH2.54 socket with VCC, TX, RX and GND (C1 specifications). Wire by the signal names printed on the board or in Slamtec's datasheet, not by pin position; third-party diagrams disagree on the order.
- The C1 draws about 230 mA. The 6C's ports other than TELEM1 share a combined 1.5 A limit (PX4 6C page), so TELEM2 power is within spec. If you see lidar dropouts, move its VCC and GND to a separate 5 V BEC and keep its ground tied to the Pixhawk's.

Raspberry Pi 5 → TELEM3

TELEM3 pin	Signal	Pi 5 header pin
1	+5 V	**Leave unconnected**
2	TX	Pin 10 (GPIO15, RXD)
3	RX	Pin 8 (GPIO14, TXD)
6	GND	Pin 6 (GND)

Pins 4 and 5 stay unconnected; on early 6C units they are I2C. Power the Pi from a dedicated 5 V 5 A BEC into its USB-C port; ArduPilot advises against powering a companion computer from a TELEM port. The ground wire to Pi pin 6 must still be connected, or the link won't work.

GPS → GPS1, receiver → RC IN

The GPS stays on GPS1 as it was under PX4. Plug the FS-iA6B's PPM output into **RC IN** with the RC cable that came with the 6C; ArduPilot detects PPM automatically.

Mounting the lidar

- Flat, with its cable exit pointing to the **rear** (ArduPilot RPLidar page).
- On the S500 the props, arms and GPS mast sit near the scan plane. Mount the C1 on a short mast above the props, with nothing else in its scan plane. Anything it sees all the time is treated as an obstacle (Phase 6 checks this).

Before powering on

- [x] ~~Every TX goes to an RX, every RX to a TX.~~
- [x] ~~No 5 V wire between the Pixhawk and the Pi.~~
- [x] ~~Ground connected Pixhawk ↔ Pi and Pixhawk ↔ lidar.~~
- [x] ~~Nothing on TELEM3 pins 4 and 5.~~
- [x] ~~Every BEC output checked with a multimeter before connecting it.~~
- [x] ~~Props off.~~

Phase 5: Base setup in QGroundControl

All in QGroundControl on Ubuntu, Pixhawk on USB. Props off. Setup pages are under **Vehicle Setup** (Vehicle Configuration in newer versions); page names can differ slightly between QGC versions. Parameters are set in **Parameters**: search the name, click it, type the value, **Save**. Reboot with **Parameters → Tools → Reboot Vehicle** when a step says so.

5.1 Frame. Frame page → Quad, X. That sets:

```
FRAME_CLASS = 1
FRAME_TYPE  = 1
```

5.2 Serial ports (Phase 4 port plan). Reboot afterwards.

```
SERIAL1_PROTOCOL = 2
SERIAL1_BAUD     = 57
SERIAL2_PROTOCOL = 11
SERIAL2_BAUD     = 460
BRD_SER2_RTSCTS  = 0
SERIAL5_PROTOCOL = 2
SERIAL5_BAUD     = 921
```

If `BRD_SER5_RTSCTS` appears, set it to 0 too.

5.3 Calibration on the **Sensors** and **Radio** pages:

1. **Accelerometer:** follow the six orientations, then **Level Horizon** with the drone flat.
2. **Compass:** rotate the drone through every axis until it finishes, then reboot.
3. **Radio:** transmitter on, receiver bound. Calibrate, moving every stick and switch to its limits.

5.4 Flight modes. Flight Modes page, on a 3-position switch:

Position	Mode	Purpose
1	Stabilize	Full manual
2	Loiter	GPS hold, your normal takeover mode
3	RTL	Come home

Flipping this switch during autonomous flight immediately replaces GUIDED. That is your manual override.

5.5 Battery monitor (6C power port 1; values from the ArduPilot 6C page), on the **Power** page or in Parameters:

```
BATT_MONITOR    = 4
BATT_VOLT_PIN   = 8
BATT_CURR_PIN   = 4
BATT_VOLT_MULT  = 18.182
BATT_AMP_PERVLT = 36.364
```

Reboot, then compare QGC's battery voltage with a multimeter on the battery. Adjust `BATT_VOLT_MULT` if they differ by more than 0.1 V.

5.6 Motor outputs (DShot300 on AUX 1–4). The ESCs stay wired exactly as under PX4, and DShot needs no ESC calibration. On the 6C, MAIN 1–8 are `SERVO1` – `SERVO8` and AUX 1–8 are `SERVO9` – `SERVO16`. Functions 33–36 are Motor 1–4, and PX4 and ArduPilot number quad X motors the same way (1 front-right, 2 rear-left, 3 front-left, 4 rear-right), so these values copy your PX4 mapping:

```
MOT_PWM_TYPE     = 5
SERVO1_FUNCTION  = 0
SERVO2_FUNCTION  = 0
SERVO3_FUNCTION  = 0
SERVO4_FUNCTION  = 0
SERVO9_FUNCTION  = 35
SERVO10_FUNCTION = 34
SERVO11_FUNCTION = 36
SERVO12_FUNCTION = 33
```

`MOT_PWM_TYPE` = 5 is DShot300. DShot600 made these ESCs pulse every 2 seconds, so don't go back to 6. Reboot, connect the flight battery (props still off), then open the **Motors** page:

1. Test each motor at low throttle. ArduPilot's letters A, B, C, D go clockwise from the front-right motor.
2. Check each letter spins the motor in that position, in the direction ArduPilot's quad X diagram shows: front-right and rear-left counter-clockwise, front-left and rear-right clockwise.
3. Wrong motor spins: swap the `SERVO9` – `SERVO12` values. Wrong direction: swap any two of that motor's three wires, or reverse it in the ESC configurator.
4. Repeat until all four are right. Disconnect the battery.

5.7 Starting tune. QGC has no equivalent of Mission Planner's Initial Tune Parameters helper. Use ArduPilot Methodic Configurator, which runs on Linux and works out the starting filter and controller values from your props and battery. Install it in its own environment inside your repo (ignored by Git, so only your parameter files get committed), and close QGC first so it can reach the USB port. Later runs need only the activate line and the last line:

```
cd $REPO
grep -qx '.venv-amc/' .gitignore || echo '.venv-amc/' >> .gitignore
python3 -m venv $REPO/.venv-amc
source $REPO/.venv-amc/bin/activate
pip install --upgrade pip
pip install ardupilot_methodic_configurator
ardupilot_methodic_configurator
```

Enter 10-inch props and a 4S battery when it asks for your components, and apply its initial values. Alternatively, boot Windows once and use Mission Planner's Initial Tune Parameters, then save the parameters to your repo.

5.8 Failsafes and fence. On the **Failsafes** page or in Parameters. Starting values for a 4S pack:

```
FS_THR_ENABLE   = 1
BATT_LOW_VOLT   = 14.0
BATT_CRT_VOLT   = 13.2
BATT_FS_LOW_ACT = 2
FS_GCS_ENABLE   = 1
FENCE_ENABLE    = 1
FENCE_TYPE      = 7
FENCE_RADIUS    = 100
FENCE_ALT_MAX   = 50
```

That is RTL on RC loss, low battery or ground-station loss, inside a 100 m radius, 50 m ceiling geofence. Copter 4.7 renamed some position parameters to metres; if a name doesn't appear, search for its `_M` variant.

Phase 5 passes when: no setup page is marked red, the motor test is right for all four motors, and the only pre-arm warning left is about proximity (fixed in Phase 6). Save `$REPO/params/s500_ardu_05_base.params` and commit.

Phase 6: Lidar and obstacle avoidance

The goal: a lidar picture that matches reality in every direction, then avoidance on with the values already proven in SITL. Props off. The serial port (TELEM2 = SERIAL2) was set in Phase 5.

6.1 Enable the lidar (ArduPilot RPLidar page):

```
PRX1_TYPE   = 5
PRX1_ORIENT = 0
```

Reboot. You should feel a slight vibration as the C1 spins up.

6.2 Check it's alive. QGC has no radar view, so use **Analyze Tools → MAVLink Inspector →** **`DISTANCE_SENSOR`**. ArduPilot reports the lidar as distance readings by direction; the `orientation` field says which: 0 front, 2 right, 4 back, 6 left (odd numbers are the diagonals). If there are no `DISTANCE_SENSOR` messages, or you get "PreArm: check

proximity sensor": check TX/RX aren't swapped, `SERIAL2_PROTOCOL` = 11, `SERIAL2_BAUD` = 460, and the C1 has 5 V at VCC. Optional: Mission Planner on Windows shows the same data as a radar picture (Ctrl+F → Proximity).

6.3 Check the directions. Hold a box or board about 1.5 m from the drone and watch which `DISTANCE_SENSOR` orientation drops to about 1.5 m:

- [x] ~~In front → shows in front.~~
- [x] ~~On the right → shows on the right.~~
- [x] ~~Behind → shows behind.~~
- [x] ~~On the left → shows on the left.~~

Everything rotated by a fixed angle means the C1's cable isn't pointing to the rear: set `PRX1_YAW_CORR` or remount. Left and right swapped means `PRX1_ORIENT` is wrong.

6.4 Mask the airframe. With nothing around, any permanent red lines are parts of the S500. For each one:

```
PRX1_IGN_ANG1 = <angle of the obstruction, degrees>
PRX1_IGN_WID1 = <width to ignore, degrees>
```

Use `IGN_ANG2` / `IGN_WID2` and so on for more. Keep them narrow: the drone is blind inside them. Remounting clear of the frame is better than masking.

6.5 Turn on avoidance (same values as SITL):

```
AVOID_ENABLE = 7
AVOID_MARGIN = 2
AVOID_BEHAVE = 1
OA_TYPE      = 1
```

Refresh the parameters (**Parameters → Tools → Refresh**); the `OA_BR_` parameters appear only after `OA_TYPE` is set. Then:

```
OA_BR_TYPE      = 1
OA_BR_LOOKAHEAD = 5
OA_MARGIN_MAX   = 2
OA_DB_ALT_MIN   = 2
GUID_OPTIONS    = 64
```

Reboot. BendyRuler won't run until the Pixhawk restarts.

Parameter	What it does
`AVOID_ENABLE` = 7, `AVOID_MARGIN` = 2, `AVOID_BEHAVE` = 1	Simple avoidance: stop 2 m from obstacles. Works **only in Loiter and AltHold**
`OA_TYPE` = 1	BendyRuler path planning in GUIDED, AUTO and RTL (BendyRuler page)
`GUID_OPTIONS` = 64	Makes GUIDED use path planning, so BendyRuler applies to your scripts' position targets
`OA_BR_LOOKAHEAD` = 5	Look 5 m ahead when planning
`OA_MARGIN_MAX` = 2	Keep 2 m from obstacles while planning
`OA_DB_ALT_MIN` = 2	Ignore obstacles near home below 2 m, so the ground isn't one during takeoff

In GUIDED, BendyRuler is the only avoidance, and only for **position targets**. Velocity commands (WASD-style) get none.

Phase 6 passes when: the radar view matches reality in all four directions, the frame is clear or masked, pre-arm checks pass, and the parameters above are set. Save `$REPO/params/s500_ardu_06_lidar.param` and commit.

Phase 7: Raspberry Pi 5 on TELEM3

The goal: the Pi reads the Pixhawk's heartbeat over TELEM3 and sends a body-relative move you've already proven in SITL. Based on ArduPilot's Raspberry Pi via MAVLink guide, with the Pi 5 differences added.

7.1 Install the OS. Raspberry Pi Imager → **Raspberry Pi OS Lite (64-bit)**. In Imager's settings set the hostname (`s500-pi`), your username, bench Wi-Fi, and enable SSH. Boot and SSH in.

7.2 Enable the GPIO UART. On the Pi 5, GPIO 14/15 is `/dev/ttyAMA0` and must be switched on; `/dev/serial0` points at the small debug connector instead.

```
sudo raspi-config
```

Interface Options → Serial Port → login shell over serial: **No** → serial port hardware: **Yes**. Then:

```
echo "dtparam=uart0=on" | sudo tee -a /boot/firmware/config.txt
sudo usermod -a -G dialout $USER
sudo reboot
```

After the reboot, `ls -l /dev/ttyAMA0` should show the port.

7.3 Software and your repo. The Pi doesn't need ArduPilot's source, and a plain clone skips the submodule. The repo is public, so the Pi clones it over HTTPS with no keys, and only ever pulls.

```
sudo apt update && sudo apt install -y python3-venv git
python3 -m venv ~/drone-venv
source ~/drone-venv/bin/activate
pip install pymavlink MAVProxy
git clone https://github.com/Antonio9381/TronV2ardrupilot.git
~/TronV2ardrupilot
```

7.4 First contact (Pixhawk on USB power, Pi on its own supply, TELEM3 wired as in Phase 4):

```
source ~/drone-venv/bin/activate
mavproxy.py --master=/dev/ttyAMA0 --baudrate 921600
```

MAVProxy should print the flight mode and load parameters. Stuck at "Waiting for heartbeat": TX/RX swapped, ground missing, `SERIAL5_PROTOCOL` / `SERIAL5_BAUD` not 2/921, or `dtparam=uart0=on` missing.

7.5 The relative-move script. Save as `$REPO/pi/nudge.py`. It finds the Pixhawk's own heartbeat, refuses unless the drone is armed and in GUIDED, and caps any move at 5 m. With `GUID_OPTIONS` = 64 these moves are planned around obstacles too.

```
import sys

from pymavlink import mavutil

mav = mavutil.mavlink

if len(sys.argv) != 5:
    sys.exit("usage: nudge.py CONN FORWARD RIGHT DOWN")
```

```
conn = sys.argv[1]
fwd, right, down = (float(v) for v in sys.argv[2:5])
if max(abs(fwd), abs(right), abs(down)) > 5:
    sys.exit("move larger than 5 m, refusing")

m = mavutil.mavlink_connection(conn, baud=921600, source_system=1,
source_component=191)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True, timeout=10)
    if hb is None:
        sys.exit("no heartbeat from the drone")
    if hb.autopilot == mav.MAV_AUTOPILOT_ARDUPILOTMEGA and
hb.get_srcComponent() == mav.MAV_COMP_ID_AUTOPILOT1:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

mode = mavutil.mode_string_v10(hb)
armed = bool(hb.base_mode & mav.MAV_MODE_FLAG_SAFETY_ARMED)
print("mode", mode, "armed", armed)
if mode != "GUIDED" or not armed:
    sys.exit("not flying in GUIDED, refusing to send")

m.mav.set_position_target_local_ned_send(
    0, m.target_system, m.target_component,
    mav.MAV_FRAME_BODY_OFFSET_NED,
    0b110111111000,
    fwd, right, down, 0, 0, 0, 0, 0, 0, 0, 0)
print("sent")
```

Test in SITL first, on the laptop. Add `--out=udp:127.0.0.1:14552` to the Phase 1.4 command, take off with `mode guided`, `arm throttle`, `takeoff 10`, then:

```
apenv
python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 0 2 0
```

The drone moves 2 m to its right. Repeat towards a post: it should route around it rather than hit it.

On the real Pi (props off, disarmed): `python3 ~/TronV2ardrupilot/pi/nudge.py /dev/ttyAMA0 0 2 0` should print the mode and refuse. That proves the link and the safety check; the real move happens in Phase 9.

Phase 7 passes when: MAVProxy on the Pi sees the Pixhawk, `nudge.py` works in SITL, and on the real Pi it reads the mode and refuses outside GUIDED.

Phase 8: Ground station (QGroundControl + MAVProxy + your scripts)

Only one program can open the SiK radio's serial port. MAVProxy opens it and shares the link: QGroundControl on UDP 14550, the old scripts on 14551, tron on 14553. This is the same arrangement SITL already uses, so the scripts don't change.

8.1 Install QGroundControl stable. Download the stable AppImage from the QGroundControl website (not Daily), then:

```
chmod +x QGroundControl-x86_64.AppImage
./QGroundControl-x86_64.AppImage
```

In QGC's **Application Settings → General → AutoConnect**, untick **SiK Radio**. Otherwise QGC grabs the radio's port before MAVProxy can. Leave **UDP** ticked.

8.2 Start the router. Plug in the ground SiK radio first, then:

```
apenv
ls /dev/ttyUSB*
mavproxy.py --master=/dev/ttyUSB0 --baudrate 57600 --out udp:127.0.0.1:14550
--out udp:127.0.0.1:14551 --out udp:127.0.0.1:14553
```

Use the device name `ls` showed. When MAVProxy shows the flight mode, open QGroundControl; it connects to UDP 14550 automatically. Keep the MAVProxy terminal open for the whole flight: closing it cuts QGC and the scripts off.

8.3 The field target script. Save as `$REPO/gcs/target.py`. Unlike `goto.py`, it never arms or takes off: it sends one target only when the drone is already flying in GUIDED.

```
import math
import sys
import time

from pymavlink import mavutil

mav = mavutil.mavlink
MAX_DIST = 100
```

```
if len(sys.argv) != 5:
    sys.exit("usage: target.py CONN LAT LON ALT")
conn = sys.argv[1]
lat, lon, alt = (float(v) for v in sys.argv[2:5])
if not 2 <= alt <= 40:
    sys.exit("alt must be 2-40 m")

m = mavutil.mavlink_connection(conn, source_system=255, source_component=190)
while True:
    hb = m.recv_match(type="HEARTBEAT", blocking=True, timeout=30)
    if hb is None:
        sys.exit("no heartbeat from the drone")
    if hb.autopilot == mav.MAV_AUTOPILOT_ARDUPILOTMEGA and
hb.get_srcComponent() == mav.MAV_COMP_ID_AUTOPILOT1:
        m.target_system = hb.get_srcSystem()
        m.target_component = hb.get_srcComponent()
        break

mode = mavutil.mode_string_v10(hb)
armed = bool(hb.base_mode & mav.MAV_MODE_FLAG_SAFETY_ARMED)
if mode != "GUIDED" or not armed:
    sys.exit(f"refusing: mode {mode}, armed {armed}")


def distance(p):
    dn = (lat - p.lat / 1e7) * 111320
    de = (lon - p.lon / 1e7) * 111320 * math.cos(math.radians(lat))
    return math.hypot(dn, de)


def recv(types, timeout=3):
    end = time.time() + timeout
    while time.time() < end:
        msg = m.recv_match(type=types, blocking=True, timeout=0.5)
        if msg and msg.get_srcSystem() == m.target_system and
msg.get_srcComponent() == m.target_component:
            return msg
    sys.exit("lost telemetry, stopping (drone keeps its last target; take
over with the mode switch)")


m.mav.command_long_send(m.target_system, m.target_component,
mav.MAV_CMD_SET_MESSAGE_INTERVAL, 0,
```

```
                        mav.MAVLINK_MSG_ID_GLOBAL_POSITION_INT, 200000, 0, 0,
0, 0, 0)
start = distance(recv("GLOBAL_POSITION_INT", 10))
if start > MAX_DIST:
    sys.exit(f"target is {start:.0f} m away, limit {MAX_DIST} m, refusing")

m.mav.set_position_target_global_int_send(
    0, m.target_system, m.target_component,
    mav.MAV_FRAME_GLOBAL_RELATIVE_ALT_INT,
    0b110111111000, int(lat * 1e7), int(lon * 1e7), alt,
    0, 0, 0, 0, 0, 0, 0, 0)

deadline = time.time() + 30 + 3 * start
while time.time() < deadline:
    msg = recv(["HEARTBEAT", "GLOBAL_POSITION_INT"])
    if msg.get_type() == "HEARTBEAT":
        mode = mavutil.mode_string_v10(msg)
        armed = bool(msg.base_mode & mav.MAV_MODE_FLAG_SAFETY_ARMED)
        if mode != "GUIDED" or not armed:
            sys.exit(f"stopped: pilot took over (mode {mode}, armed
{armed})")
        continue
    d = distance(msg)
    print(f"{d:6.1f} m to go, alt {msg.relative_alt / 1000:5.1f} m")
    if d < 2:
        print("arrived")
        break
else:
    sys.exit("timed out; take over with the mode switch")
```

Test it in SITL first: take off manually in MAVProxy (`mode guided`, `arm throttle`, `takeoff 10`), then, in another terminal:

```
apenv
python3 $REPO/gcs/target.py udpin:0.0.0.0:14551 <lat> <lon> 10
```

In the field the command is identical, with the router from 8.2 running.

8.4 Where the agent fits. Once Phase 9 passes, the LLM planner goes in front of these scripts: sentence → objective dictionary → validated in plain code (geofence, altitude, known targets) → `target.py`-style GPS targets from the ground station, and `nudge.py`-style relative moves from the Pi's mission manager. QGroundControl stays on screen the whole time as your view and override.

Phase 8 passes when: MAVProxy holds the radio link, QGC shows the drone through it, and `target.py` works against SITL and refuses on the real drone while disarmed.

Phase 9: Flight tests

Each flight adds exactly one new thing; move on only when the previous one passes. Fly in an open field, within CASA's rules and your club's field rules, with a second person watching.

Every flight: one hand near the mode switch. Anything unexpected → **Loiter** (or Stabilize if GPS is the problem), land, read the log before flying again.

1. **Manual hover.** Props on for the first time. Take off in Stabilize, hover at 2–3 m, then try Loiter. *Passes:* no oscillation, holds position in Loiter, all four motors balanced in the log.
2. **Failsafes.** At 10 m in Loiter, switch to RTL: it comes home and lands. Then, with a spotter, turn the transmitter off briefly: RC failsafe RTL. Switch it back on and retake control with the mode switch. *Passes:* both RTLs behave, and you can always retake control.
3. **Lidar in Loiter.** Fly slowly towards a large soft obstacle (foam board, inflatable, a wall with plenty of room). *Passes:* stops about 2 m away however you push the stick.
4. **Go to location from QGC.** Take off in Loiter to 10 m. In QGC, click the flight mode in the top toolbar and choose **Guided**, then click the map 30–50 m away with nothing in between → **Go to location** and confirm the slider. *Passes:* flies there and holds.
5. **Go to location past one obstacle.** Lower the waypoint speed first (`WP_SPD, about 2 m/s; yours is 3`). Put the obstacle on the line. *Passes:* bends around it, keeping about 2 m away.
6. **Target from your script.** Same as 5, but with `target.py` through the MAVProxy router (Phase 8). *Passes:* "arrived", with avoidance.
7. **Pi relative moves.** Hover in GUIDED at 10 m, SSH into the Pi from close range, run `nudge.py /dev/ttyAMA0 0 2 0`. *Passes:* moves about 2 m right; repeated towards the obstacle, it routes around it.
8. **Ground station link loss in GUIDED.** Unplug the ground radio during a target flight. *Passes:* RTL after the GCS failsafe timeout.
9. **Only now: the agent.** Start the mission manager and planner with simple objectives (one target, then target plus search) before adding camera tracking.

After each passing flight, download the log (QGC: **Analyze Tools → Log Download**) into `$REPO/logs/` with that day's parameter file. For plots, open the `.bin` log in UAV Log Viewer (plot.ardupilot.org) or MAVExplorer.

Once the basics are solid, run AutoTune at a safe height for a better-tuned S500, especially before adding a camera payload.

Testing every script

Run the tests in this order: unit tests, then SITL, then the Pi on the bench, then the real drone on the ground, then flights. Don't move on until the current test passes. Each test lists every terminal from a fresh start, so you can jump straight to one.

Test	Script	Where	Props	Passes when
1	tron unit tests	Laptop	n/a	64 passed
2	SITL session	Laptop	n/a	Console shows GUIDED-ready, 3 outputs
3	gcs/goto.py	Laptop + SITL	n/a	"arrived", bends round posts
4	gcs/target.py	Laptop + SITL	n/a	Refuses on the ground, "arrived" in the air
5	pi/nudge.py	Laptop + SITL	n/a	Moves 2 m right
6	tron	Laptop + SITL	n/a	Every row of the Test 6 table matches
7	Old + tron together	Laptop + SITL	n/a	Both work in one session
8	tron on the Pi	Pi	n/a	64 passed, camera sees the colour
9	All scripts, ground	Real drone	Off	Every script reads the drone and refuses
10	All scripts, flying	Real drone	On	Each old/new pair does the same move

Terminal names used below: **L1, L2, L3** are laptop terminals, **P1, P2** are SSH sessions into the Pi. `apenv` is your alias that activates the ArduPilot venv (Phase 1.1); `$REPO` is your repo on the laptop.

Port map. Every script listens on its own UDP port, so they never fight over one connection.

Port	Who listens	Used by
14550	QGroundControl	You, watching
14551	goto.py, target.py	Old ground scripts
14552	nudge.py (SITL only)	Old Pi script, tested on the laptop
14553	tron	New code on the laptop
14560	tron	New code on the Pi
14561	nudge.py	Old Pi script on the Pi

Test 1: Install tron and run its unit tests (laptop)

This needs no drone and no simulator. The tests fly a simulated drone and a fake autopilot that talks real MAVLink.

1. Put the package in the repo. Save `tron-update.zip from the chat into ~/Downloads. It adds agent/ and replaces gcs/goto.py, gcs/target.py and pi/nudge.py with the checked versions printed in this guide. I`n **L1**:

```
cd $REPO
unzip -o ~/Downloads/tron-update.zip
git status
ls agent
```

git status should list agent/ as new and the three scripts as modified. ls agent should show `tron`, `tests`, `pyproject.toml`, `README.md` and two `.toml` configs.

2. Install it into the ArduPilot venv, so `tron` shares the same pymavlink as your old scripts:

```
apenv
cd $REPO/agent
pip install -e '.[test]'
which tron
```

`which tron` should print a path inside the venv. `-e` means edits to the code take effect without reinstalling.

3. Run the tests:

```
cd $REPO/agent
pytest
```

It takes about 35 seconds. **Passes when:** the last line says `64 passed`. Anything `failed` → stop and send me the output.

4. Check the command line works:

```
tron --version
echo '{"lat": 51.8754761, "long": 14.6490749, "alt": 10, "colour": "red"}' >
/tmp/obj.json
tron check-objective /tmp/obj.json
echo '{"lat": 51.87, "alt": 10}' > /tmp/bad.json
tron check-objective /tmp/bad.json; echo "exit $?"
```

The first prints the parsed objective. The second logs `objective error: missing long` and `exit 2`.

5. Keep run logs out of Git, then commit and push so the Pi can pull it:

```
cd $REPO
grep -qx 'agent/logs/' .gitignore || echo 'agent/logs/' >> .gitignore
git add .gitignore agent gcs pi
git commit -m "Add tron package, harden goto/target/nudge"
git push
```

Test 2: Start one SITL session for all script tests

Tests 3 to 7 all use this one simulator. It is the Phase 1.4 command with two extra outputs, so every script gets its own port.

1. **L1**, start SITL (no `-w`, so your Phase 1 settings stay):

```
apenv
cd $REPO/ardupilot/ArduCopter
../Tools/autotest/sim_vehicle.py -v ArduCopter -A --serial5=sim:rplidara2 -l
51.8752066,14.6487840,54.15,0 --console --map --out=udp:127.0.0.1:14551 --
out=udp:127.0.0.1:14552 --out=udp:127.0.0.1:14553
```

2. In the same terminal, at the `STABILIZE>` prompt, one line at a time:

```
script /tmp/post-locations.scr
param show GUID_OPTIONS
param show OA_TYPE
param set SIM_BATT_VOLTAGE 16.8
```

`GUID_OPTIONS` must say 64 and `OA_TYPE` 1. If not, redo Phase 1.5. `SIM_BATT_VOLTAGE` makes the simulated battery a full 4S. SITL's default is 12.6 V, and tron refuses to fly below 14.2 V.

3. Wait until the console shows **GPS: OK** and **EKF** is no longer red (about 30 seconds). The map shows the drone at the test field with the blue posts.

Take off by hand (used by Tests 4, 5 and 7), at the `STABILIZE>` prompt, one line at a time:

```
mode guided
arm throttle
takeoff 10
```

Reset between tests: type `mode rtl` and wait for it to land and disarm. Quit SITL with Ctrl+C in L1 when you're done.

Picking a target: move the mouse over the map; the bottom of the map window shows the latitude and longitude under it. Points used below, from the SITL home (51.8752066, 14.6487840):

Name	Lat	Lon	Where
A	51.8754761	14.6490749	30 m north, 20 m east
B	51.8752066	14.6491477	25 m east
FAR	51.8758354	14.6487840	70 m north
OUT	51.8770032	14.6487840	200 m north, outside tron's 80 m limit

Test 3: goto.py in SITL

`goto.py` arms, takes off and flies to a point by itself, but only after you put the drone in GUIDED. These SITL checks must pass before its field test in Test 10.

1. SITL running from Test 2, drone **landed and disarmed**, mode **STABILIZE** (type `mode stabilize` if not).
2. **Refusal checks, L2**, one line at a time:

```
apenv
cd $REPO/gcs
python3 goto.py udpin:0.0.0.0:14551 51.8754761 14.6490749 10
python3 goto.py udpin:0.0.0.0:14551 51.8770032 14.6487840 10
python3 goto.py udpin:0.0.0.0:14551 51.8754761 14.6490749 60
```

Expect, in order: `not in GUIDED, refusing ...`, then `target is 200 m away, limit 100 m, refusing`, then `alt must be 2-30 m`. The drone never arms.

3. **The flight.** In L1 type `mode guided` (the drone stays disarmed on the ground). Then in **L2**:

```
python3 goto.py udpin:0.0.0.0:14551 51.8754761 14.6490749 10
```

It prints `mode GUIDED, armed False, target 36 m away`, `armed`, `taking off to 10 m`, then counts down `... m to go, alt 10.0 m` and ends with `arrived`. On the map it climbs, then flies to point A round any post.

4. **Already flying.** While it hovers at A, send it to B:

```
python3 goto.py udpin:0.0.0.0:14551 51.8752066 14.6491477 10
```

It prints `armed True`, skips the takeoff, and flies straight to B.

5. **Override.** Send it to FAR, and about 5 seconds in type `rc 3 1500` then `mode loiter` in L1:

```
python3 goto.py udpin:0.0.0.0:14551 51.8758354 14.6487840 10
```

It must print `stopped: pilot took over (mode LOITER, armed True)` and exit; the drone holds in Loiter.

6. **Passes when:** all three refusals in step 2, `arrived` in steps 3 and 4, and the stop in step 5.

7. Reset: `mode rtl` in L1.

It prints `no heartbeat from the drone` after 30 s if it can't hear SITL: check the L1 command has `--out=udp:127.0.0.1:14551`. If arming is refused, the reason is on the `FC:` lines above it.

Test 4: target.py in SITL

`target.py` never arms or takes off; it only sends a target when the drone is already flying in GUIDED. This test checks both halves.

1. SITL running from Test 2, drone **landed and disarmed**.
2. **Refusal check, L2:**

```
apenv
cd $REPO/gcs
python3 target.py udpin:0.0.0.0:14551 51.8754761 14.6490749 10
```

It must print `refusing: mode STABILIZE, armed False` (or whatever mode it's in) and exit. The drone doesn't move.

3. Take off by hand in L1: `mode guided`, `arm throttle`, `takeoff 10`. Wait until the console shows about 10 m.
4. **L2**, same command again:

```
python3 target.py udpin:0.0.0.0:14551 51.8754761 14.6490749 10
```

Still hovering at A, two more checks in **L2**:

```
python3 target.py udpin:0.0.0.0:14551 51.8770032 14.6487840 10
python3 target.py udpin:0.0.0.0:14551 51.8758354 14.6487840 10
```

The first must print `target is 172 m away, limit 100 m, refusing` (measured from where the drone is). The second flies toward FAR: about 5 seconds in, type `rc 3 1500` then `mode loiter` in L1. It must print `stopped: pilot took over (mode LOITER, armed True)`. Type `mode guided` in L1 afterwards.

5. **Passes when:** step 2 refused, the 172 m target was refused, the Loiter switch stopped it, and step 4 counts down `m to go, alt 10.0 m` and ends with `arrived` at point A.
6. Reset: `mode rtl` in L1.

Test 5: nudge.py in SITL

`nudge.py` is the Pi's relative-move script. On the laptop it listens on 14552 instead of the Pi's serial port, and otherwise runs exactly as on the Pi.

1. SITL running from Test 2, drone **landed and disarmed**.
2. **Refusal check, L2:**

```
apenv
python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 0 2 0
```

It prints `mode STABILIZE armed False` (or the current mode) then `not flying in GUIDED, refusing to send`.

3. **Size check, L2:**

```
python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 6 0 0
```

It prints `move larger than 5 m, refusing` straight away.

4. Take off by hand in L1: `mode guided`, `arm throttle`, `takeoff 10`.
5. **L2**, the real move:

```
python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 0 2 0
```

It prints `mode GUIDED armed True` then `sent`. On the map the drone moves about 2 m to its right (east, if its nose points north).

6. Try `3 0 0` (3 m forward) and `0 0 -2` (2 m up; negative down is up).
7. **Passes when:** steps 2 and 3 refused, and each move in 5 and 6 went the right direction by about the right distance.
8. Reset: `mode rtl` in L1.

Test 6: tron in SITL

This runs every tron command and trips every safety stop on purpose. tron listens on 14553. It prints one result line, and its exit code is 0 for done, 1 for stopped part way, 2 for refused before sending anything.

1. SITL running from Test 2, drone **landed and disarmed**, `SIM_BATT_VOLTAGE` 16.8.
2. **L3**, make a SITL config with a smaller search square so the search test finishes in about 2 minutes:

```
apenv
cd $REPO/agent
printf '[link]\nconn = "udpin:0.0.0.0:14553"\n\n[mission]\nsearch_radius_m =
10\n' > config.sitl.toml
alias t='tron -c config.sitl.toml'
```

3. **Status:**

```
t status
```

Prints JSON: `mode`, `armed: false`, `sats` 10 or more, `battery_v` 16.8, and `home` near 51.8752, 14.6488. If it says `no ArduPilot heartbeat`, L1 is missing `--out=udp:127.0.0.1:14553`.

4. **Refuses on the ground:**

```
t goto 51.8754761 14.6490749 10; echo "exit $?"
```

Expect `rejected mode is STABILIZE, need GUIDED` (or whatever mode it's in) and `exit 2`. Nothing is sent.

5. **Take off** (works only in SITL; it checks for the simulator first):

```
t sitl-takeoff 10
```

The drone arms in GUIDED and climbs; the command returns when it reaches about 10 m.

6. **Goto and nudge:**

```
t goto 51.8754761 14.6490749 10
t nudge 0 3
t nudge 2 0 -2
t status
```

Each prints `arrived`. The second nudge goes 2 m forward and 2 m up, so `status` shows `rel_alt` about 12.

7. **Limits** (each must print `rejected ...`, exit 2, and the drone must not move):

```
t goto 51.8770032 14.6487840 10
t goto 51.8754761 14.6490749 48
t nudge 6 0
```

Expect `target 200 m from home, limit 80 m`, `alt 48.0 m outside 3-40 m` and `nudge larger than 5.0 m`.

8. **RC override.** Start a long goto in L3:

```
t goto 51.8758354 14.6487840 10
```

While it flies, in L1 type `rc 3 1500` then `mode loiter`. L3 must print `override pilot took over (LOITER, armed=True)` at once, and the drone holds in Loiter. Back in L1: `mode guided`.

9. **Ctrl+C.** Run the same goto again, and press Ctrl+C in L3 halfway. It logs `interrupted, holding position` and the drone stops where it is.
10. **Low battery.** Start a goto back over home:

```
t goto 51.8752066 14.6487840 10
```

While it flies, in L1: `param set SIM_BATT_VOLTAGE 14.0`. L3 prints `low_battery battery 14.00 V` and the drone holds. Restore it: `param set SIM_BATT_VOLTAGE 16.8`.

If SITL's own battery failsafe switches to RTL first, L3 prints `override` instead. That's also a pass: either way tron stopped.

11. **Mission, no colour** (fly there and hold):

```
echo '{"lat": 51.8752066, "long": 14.6491477, "alt": 10}' > /tmp/obj_b.json
t mission /tmp/obj_b.json
```

Expect `mission ok: phase=transit found=False arrived`.

12. **Mission with a colour.** SITL has no camera, so it flies the whole search square, finds nothing and holds:

```
echo '{"lat": 51.8754761, "long": 14.6490749, "alt": 10, "colour": "red",
"target": "tarp"}' > /tmp/obj_red.json
t mission /tmp/obj_red.json
```

On the map it flies to A, then an expanding square of 9 points, 5 m apart, pausing 2 s at each. Expect `mission failed: phase=search found=False arrived search finished without a sighting`. That proves the search pattern; finding a target needs the real camera (Test 8).

13. **Finish:**

```
t rtl
```

The drone returns and lands.

Passes when: every step printed what it says above. The full log of each run is in `$REPO/agent/logs/`, one file per command, newest last (`ls -t logs | head`).

Test 7: Old scripts and tron side by side in SITL

This is the rehearsal for Test 10: the same moves from both, in one session, taking turns. **Only one script commands at a time**: wait for one to finish before starting the other. `status` is safe to run any time.

1. SITL running from Test 2, landed and disarmed. L2 in `$REPO/gcs` with `apenv`; L3 in `$REPO/agent` with `apenv` and the `t` alias from Test 6.
2. Take off by hand in L1: `mode guided`, `arm throttle`, `takeoff 10`.
3. Run these pairs, one line at a time, waiting for each to finish:

Step	Old (L2)	New (L3)	Both should
a	`python3 target.py udpin:0.0.0.0:14551 51.8754761 14.6490749 10`		fly to A, `arrived`
b		`t goto 51.8752066 14.6491477 10`	fly to B, `arrived`
c	`python3 target.py udpin:0.0.0.0:14551 51.8752066 14.6487840 10`		fly over home
d		`t goto 51.8754761 14.6490749 10`	fly to A
e	`python3 $REPO/pi/nudge.py udpin:0.0.0.0:14552 0 2 0`		move 2 m right
f		`t nudge 0 2`	move 2 m right again
g		`t status`	read the drone while nothing else commands

4. **Passes when:** every row did what the last column says, and tron's `status` agrees with the console in L1.
5. Reset: `t rtl`.

Test 8: tron on the Pi (install, unit tests, camera)

On the bench with home Wi-Fi; no drone needed. tron gets its own venv on the Pi so the camera libraries from apt are visible to it. Your old `~/drone-venv` stays as it is.

1. **P1**, get the code (you pushed it at the end of Test 1):

```
ssh wrylands@joeyberry.local
cd ~/TronV2ardrupilot
git pull
ls agent
```

2. **P1**, install the camera libraries and tron, once:

```
sudo apt update
sudo apt install -y python3-opencv python3-numpy python3-picamera2
python3 -m venv --system-site-packages ~/tron-venv
source ~/tron-venv/bin/activate
pip install -e ~/TronV2ardrupilot/agent pytest
```

Don't `pip install opencv` into this venv: the apt one must win, or the Pi camera library breaks.

3. **P1**, unit tests:

```
cd ~/TronV2ardrupilot/agent
pytest
```

Passes when: `64 passed` (about a minute on the Pi).

4. **P1**, check the camera is seen:

```
rpicam-hello --list-cameras
```

It lists the camera module. Note which one: Camera Module 2 has a 62.2° horizontal view (the default in `config.pi.toml`), Camera Module 3 about 66°. If it differs, add `hfov_deg = 66` under `[camera]` in `config.pi.toml`. A USB webcam instead: set `source = "opencv"`.

5. **P1**, colour bench check. Point the camera at the floor, then hold something red in view:

```
cd ~/TronV2ardrupilot/agent
python3 - <<'EOF'
import time
from tron.detect import ColourDetector, Locator, PiCamera
loc = Locator(PiCamera(), ColourDetector("red"), 62.2)
for _ in range(40):
    print(loc.locate(2.0))
    time.sleep(0.5)
loc.close()
EOF
```

It prints `(forward_m, right_m)` as if the camera were 2 m up, or `None` when it sees nothing red. Move the object toward the top of the picture: forward grows. Move it right: right grows.

6. **Passes when:** tests pass, `None` with nothing red in view, and the numbers follow the object the right way. If forward or right is reversed, the camera is mounted rotated; tell me how it sits and I'll add a rotation setting.

Push any config change from the Pi only if you set up credentials there; otherwise make the same edit on the laptop and `git pull` on the Pi.

Test 9: Ground checks on the real drone (props off)

Every script connects to the real drone, reads it correctly, and refuses to move it. Do this at home the day before, and again at the field before the first takeoff.

Setup: props off, flight battery in, drone outside or near a window for GPS, disarmed. Pi and laptop on the same Wi-Fi (or the phone hotspot at the field).

1. **L1**, the radio router with the extra port for tron:

```
apenv
ls /dev/ttyUSB*
mavproxy.py --master=/dev/ttyUSB0 --baudrate 57600 --out udp:127.0.0.1:14550
--out udp:127.0.0.1:14551 --out udp:127.0.0.1:14553
```

Then open QGroundControl; it connects on 14550.

2. **L2**, old ground script:

```
apenv
cd $REPO/gcs
python3 target.py udpin:0.0.0.0:14551 0 0 10
```

Must print `refusing: mode LOITER, armed False` (or your current mode). The 0 0 target is never sent.

3. **L3**, tron on the laptop (the default config already listens on 14553):

```
apenv
cd $REPO/agent
tron status
```

Compare with QGC: same mode, `sats` matches, `battery_v` within 0.1 V, `home` where the drone is. Then:

```
tron goto <lat from status> <lon from status> 10; echo "exit $?"
tron nudge 2 0; echo "exit $?"
tron sitl-takeoff 5; echo "exit $?"
```

The first two print `rejected mode is LOITER, need GUIDED`, exit 2. The third logs `SIM_SPEEDUP not found: this is not SITL, refusing to arm`, exit 2. Also read the `limits:` log line: it must end `path planning True`, and show `radius 80 m, alt 3-40 m` from your fence. If it says `path planning False`, set `GUID_OPTIONS` = 64 and `OA_TYPE` = 1 and reboot.

Old goto.py, L2, using the `lat` and `lon` that `tron status` just printed:

```
python3 goto.py udpin:0.0.0.0:14551 <lat> <lon> 5
```

Must print `mode LOITER, armed False, target 0 m away`, then `not in GUIDED, refusing ...`. With CH6 low it can never arm the drone.

4. **P1**, on the Pi, MAVProxy owns the Pixhawk's serial port and shares it on two UDP ports. `--source-system 1` stops this MAVProxy pretending to be a ground station, so it can't hide a real loss of your radio link from the GCS failsafe.

```
ssh wrylands@joeyberry.local
source ~/drone-venv/bin/activate
mavproxy.py --master=/dev/ttyAMA0 --baudrate 921600 --source-system 1 --
source-component 192 --out udp:127.0.0.1:14560 --out udp:127.0.0.1:14561
```

Wait until it shows the flight mode. Leave it running.

5. **P2**, a second SSH session, old Pi script through the split:

```
ssh wrylands@joeyberry.local
source ~/tron-venv/bin/activate
python3 ~/TronV2ardrupilot/pi/nudge.py udpin:127.0.0.1:14561 0 2 0
```

Must print the mode, then `not flying in GUIDED, refusing to send`.

6. **P2**, tron on the Pi:

```
cd ~/TronV2ardrupilot/agent
tron -c config.pi.toml status
tron -c config.pi.toml nudge 0 2; echo "exit $?"
```

`status` matches QGC; the nudge prints `rejected ...`, exit 2.

7. **Passes when:** all five scripts read the drone, and every move was refused. Leave L1, QGC and P1 running if you're flying next.

Test 10: Field flights, old script then tron

Each move is flown twice: first with the old script you already trust, then with tron. Run the Flight-day checklist steps 1–4 and Test 9 first, and keep L1, QGC and P1 running. A spotter watches; your hand stays on the mode switches. **CH6 low** (back to Loiter) stops any script instantly.

Throttle stick at mid before you flip CH6 low in the air. In Loiter the throttle stick sets climb rate, so a stick at the bottom makes it descend the moment you take over.

Every old script stops itself when you leave GUIDED, but the last target it sent stays in the flight controller until you do. `nudge.py` returns as soon as it has sent; wait until the drone has stopped (QGC speed near 0) before the next move.

Flight 1: goto and nudge

1. Arm, take off in Loiter to about 10 m, and check it holds still hands-off for 10 seconds.
2. Flip **CH6 high**: GUIDED.
3. **L3**, read where it is and work out a point 10 m north:

```
tron status
python3 -c "from tron.geo import offset_latlon as o; print(*o(<lat>, <lon>,
10, 0))"
```

Put the `lat` and `lon` from `status` into the second line. It prints the target's lat and lon. Call the start S and the target N.

4. **L2, old:** `python3 target.py udpin:0.0.0.0:14551 <N lat> <N lon> 10`. Passes: flies 10 m north, `arrived`.
5. **L3, new:** `tron goto <S lat> <S lon> 10`. Passes: flies back, `arrived`, exit 0.
6. **P2, old:** `python3 ~/TronV2ardrupilot/pi/nudge.py udpin:127.0.0.1:14561 0 2 0`. Passes: about 2 m right.
7. **P2, new:** `tron -c config.pi.toml nudge 0 -2`. Passes: about 2 m left, `arrived`.
8. **Refusal in the air, L3:** work out a point 200 m north (`o(<lat>, <lon>, 200, 0)`) and `tron goto` it. Passes: `rejected target 200 m from home, limit 80 m`, and the drone doesn't move.
9. **Override, L3:** make a point 30 m north and `tron goto` it. After it starts moving, flip **CH6 low**. Passes: it holds in Loiter at once and L3 prints `override pilot took over (LOITER, armed=True)`.
10. Land in Loiter, disarm, battery out. Download the log.

Flight 2: past an obstacle. Same as Phase 9 item 5, but run the move as step 4 with `target.py`, then step 5 with `tron goto`, the obstacle on the line both times. Passes: both bend round it at about 2 m.

Flight 2b: nudge.py in detail (Pi). Hover at 10 m in GUIDED with at least 10 m clear all round. In **P2** (P1's MAVProxy split running), one line at a time, waiting for the drone to stop each time:

```
source ~/tron-venv/bin/activate
cd ~/TronV2ardrupilot/pi
python3 nudge.py udpin:127.0.0.1:14561 3 0 0
python3 nudge.py udpin:127.0.0.1:14561 -3 0 0
python3 nudge.py udpin:127.0.0.1:14561 0 3 0
python3 nudge.py udpin:127.0.0.1:14561 0 -3 0
python3 nudge.py udpin:127.0.0.1:14561 0 0 -2
python3 nudge.py udpin:127.0.0.1:14561 0 0 2
python3 nudge.py udpin:127.0.0.1:14561 6 0 0
```

Command	Should
`3 0 0`	go 3 m the way the nose points
`-3 0 0`	come back 3 m

Command	Should
`0 3 0`	go 3 m to its right
`0 -3 0`	come back
`0 0 -2`	climb 2 m (QGC altitude about 12 m); negative down is up
`0 0 2`	descend 2 m, back to about 10 m
`6 0 0`	print `move larger than 5 m, refusing`, no movement

Then flip **CH6 low** and run `python3 nudge.py udpin:127.0.0.1:14561 0 2 0`: it must print `mode LOITER armed True` and `not flying in GUIDED, refusing to send`. Flip CH6 high again. Last, face the Flight 2 obstacle from about 6 m and send `5 0 0`: it must stop or route round, never closer than about 2 m. **Passes when:** every row matches and nothing moved on the two refusals.

Flight 3: goto.py with automatic takeoff. Fly this only after Flights 1 and 2 pass. `goto.py` arms and takes off by itself, so the area must be clear for 20 m round both the takeoff spot and the target, with the spotter ready.

1. Drone disarmed on the ground, CH5 on Loiter, **CH6 low**, throttle stick at mid. L1, QGC and L3 running.
2. **L3**, work out a target 15 m north of the drone:

```
tron status
python3 -c "from tron.geo import offset_latlon as o; print(*o(<lat>, <lon>,
15, 0))"
```

Call the drone's position S and the printed point T.

3. **L2, ground refusal:** `python3 goto.py udpin:0.0.0.0:14551 <T lat> <T lon> 5`. With CH6 low it must print `not in GUIDED, refusing ...` and never arm.
4. Flip **CH6 high** on the ground. QGC shows Guided; the drone stays disarmed. Everyone stands back.
5. **L2, the flight:** run the same command again. It prints `armed`, `taking off to 5 m`, climbs to 5 m, flies 15 m north, counts down and prints `arrived`, then hovers. If anything looks wrong, at any moment: CH6 low (Loiter). If it's still on the ground after arming, CH6 low and it disarms by itself within seconds.

6. **L2, already flying:** `python3 goto.py udpin:0.0.0.0:14551 <S lat> <S lon> 5`. It prints `armed True`, skips the takeoff and flies back to S.
7. **Override:** make a point 30 m north (`o(<lat>, <lon>, 30, 0)`), send `goto.py` there, and flip CH6 low halfway. It must print `stopped: pilot took over (mode LOITER, armed True)` while the drone holds in Loiter.
8. Land in Loiter, disarm, battery out.

Passes when: it refused with CH6 low, took off and arrived on its own, flew back without a second takeoff, and stopped on the switch.

Flight 4: the colour mission (tron on the Pi with its camera; there is no old equivalent).

1. Lay a red tarp (at least 1 m across) about 10 m north of the takeoff spot.
2. Before takeoff, **P2**, write the objective at the tarp. Use the drone's position from `status` as the start:

```
cd ~/TronV2ardrupilot/agent
tron -c config.pi.toml status
python3 -c "from tron.geo import offset_latlon as o; print(*o(<lat>, <lon>,
10, 0))"
echo '{"lat": <tarp lat>, "long": <tarp lon>, "alt": 8, "colour": "red",
"target": "tarp"}' > ~/tarp.json
tron check-objective ~/tarp.json
```

3. Take off in Loiter to 8 m, CH6 high.
4. **P2:** `tron -c config.pi.toml mission ~/tarp.json`.
5. Watch: it flies to the point, searches if the tarp isn't under it, then follows the tarp for 30 s and holds.
6. **Passes when:** it ends with `mission ok: phase=track found=True`. Then CH6 low, land.

If a step fails, land and send me the matching log from `$REPO/agent/logs/` or `~/TronV2ardrupilot/agent/logs/`, plus the `.bin` flight log.

Flight-day checklist

The same steps every time you fly, in order. Print it or keep it open on your phone at the field.

1. At home, the day before

- [ ] Charge: flight batteries, transmitter, laptop, and the Pi's supply if it has its own pack.
- [ ] Laptop: `cd $REPO && git pull`, so you have the latest scripts.
- [ ] Pi, while it still has home Wi-Fi: `cd ~/TronV2ardrupilot && git pull`.
- [ ] Pi network for the field: there's no home Wi-Fi there, so let the Pi join your phone's hotspot. Set this up once, at home, with the hotspot switched on: `sudo nmcli dev wifi connect "<hotspot name>" password "<hotspot password>"`. The laptop joins the same hotspot at the field, so you can SSH in.
- [ ] Optional dress rehearsal: run the day's script against SITL (Phase 1.4 command, then the script on `udpin:0.0.0.0:14551`).

2. At the field: drone and transmitter

- [ ] Props on, each on the correct motor (front-right and rear-left counter-clockwise, front-left and rear-right clockwise), and tight.
- [ ] Battery strapped down. Lidar mast clear, GPS arrow pointing forward, nothing loose near the props.
- [ ] Transmitter switches **before** power: CH5 on Stabilize or Loiter, **CH6 (GUIDED) low**, **CH7 (arm) disarmed**, **CH8 (emergency stop) low**.
- [ ] **Transmitter on first**, then plug in the flight battery. Leave the drone still on level ground for about 30 seconds while the sensors initialise.

3. Laptop: terminal 1, the radio router

Plug the ground SiK radio into the laptop, then:

```
apenv
ls /dev/ttyUSB*
mavproxy.py --master=/dev/ttyUSB0 --baudrate 57600 --out udp:127.0.0.1:14550
--out udp:127.0.0.1:14551 --out udp:127.0.0.1:14553
```

Use the device `ls` showed. Wait until MAVProxy shows the flight mode and loads parameters. Leave this terminal open for the whole flight: closing it cuts off QGC and the scripts.

4. Laptop: QGroundControl

Start the stable AppImage. It connects through UDP 14550 by itself. Check in this order:

- [ ] Top bar says **Ready**, not "Not Ready". If not, read the pre-arm messages; don't override them.
- [ ] GPS: **10+ satellites**, HDOP under about 1.0, and the home **H** on the map where the drone is.
- [ ] Battery voltage about right for a charged 4S pack (16.4–16.8 V).
- [ ] Heading: the drone's arrow on the map points the way its nose points.
- [ ] Lidar: **Analyze Tools → MAVLink Inspector →** `DISTANCE_SENSOR` readings look sensible, with nothing closer than you expect.
- [ ] Flight mode shown matches your CH5 switch, and flipping CH6 changes it to GUIDED and back. Leave CH6 low afterwards.

5. Laptop: terminal 2, the Pi (if this flight uses it)

```
ssh wrylands@joeyberry.local
source ~/tron-venv/bin/activate
cd ~/TronV2ardrupilot
```

Either run Pi scripts straight on /dev/ttyAMA0 with nothing else on it, or start the Test 9 MAVProxy split first and point every Pi script at its UDP port. Never a script on `/dev/ttyAMA0` while MAVProxy has it: only one program can hold the port.

6. Laptop: terminal 3, the ground station script (if this flight uses it)

```
apenv
cd $REPO/gcs
```

Have the command typed and ready, e.g. `python3 target.py udpin:0.0.0.0:14551 <lat> <lon> 15`. Don't press Enter until the drone is hovering in GUIDED.

7. Flying a mission

1. Spotter in place, area clear, everyone behind you.
2. **Safety switch LED solid (with BRD_SAFETY_DEFLT = 0 it starts solid; if it flashes, hold the button until solid)**, then **arm** with CH7.
3. Take off in **Stabilize** or **Loiter**, and climb to about 10 m. Check it holds position steadily in Loiter.
4. Flip **CH6 high**: the mode becomes **GUIDED**.

5. Start the script (terminal 3, or terminal 2 for the Pi). Keep one hand on the switches throughout.
6. When it reports arrived or finishes, flip **CH6 low**: you're back in Loiter. Land.
7. **Disarm** with CH7, then unplug the flight battery.

If anything looks wrong:

Situation	Do
Script misbehaving, drone wandering	Throttle stick to mid, then **CH6 low**: instantly back to manual Loiter
You're unsure where it is or it's drifting away	CH5 to **RTL**
About to hit someone or something	**CH8 emergency stop**: motors cut, the drone falls. Last resort only
Script needs stopping	Old scripts: Ctrl+C stops the script, but the drone keeps flying to the last target, so flip CH6 low. tron: Ctrl+C makes it hold position

8. After the flight

- [ ] Battery unplugged before you stop anything on the laptop.
- [ ] Stop the scripts and MAVProxy with Ctrl+C.
- [ ] Download the log: QGC **Analyze Tools → Log Download**, newest log, saved into `$REPO/logs/`.
- [ ] If you changed any parameters, save them to `$REPO/params/` with the date and what changed.
- [ ] Commit and push: `cd $REPO && git add params gcs pi agent && git commit -m "Flight YYYY-MM-DD: what you tested" && git push`.
- [ ] Write one line in your notes on what worked and what to change. Then tick the flight off in Phase 9.

Troubleshooting

Everything that has come up so far, and the fix.

Symptom	Cause	Fix
`git push` rejected, "fetch first"	GitHub repo was created with its own commit (README)	`git pull origin main --allow-unrelated-histories`, resolve any conflict, then `git push -u origin main`
Install script stalls on wxPython	extras.wxpython.org not responding	Ctrl+C; wxPython 4.2.1 is already installed system-wide
`No such file or directory: 'mavproxy.py'`	ArduPilot venv not active	`apenv` before `sim_vehicle.py`
`apenv: command not found`	Alias not loaded in that terminal	`. ~/.bashrc`
`[ble: detached]`, garbled prompt after a build	Prompt add-on confused by long output	Close the terminal and open a new one
Prompt shows `detached at APMrover2-stable`	Another tag on the same commit	Check `git -C $REPO/ardupilot tag --points-at HEAD` lists Copter-4.7.2
`can't convert ... to float`, `Usage: takeoff ...`	Several MAVProxy lines pasted at once	Type one line at a time
`Unable to find parameter 'OA_BR_LOOKAHEAD'`	Set before `param fetch` finished	Wait for `Received ... parameters`, then set it
`PreArm: BendyRuler OA requires reboot`	`OA_TYPE` changed	`reboot` (SITL) or reboot the Pixhawk
Drone flies straight through obstacles in GUIDED	`GUID_OPTIONS` not 64	`GUID_OPTIONS` = 64, reboot
Settings gone after restarting SITL	Started with `-w`	Never use `-w` after the first run
QGC Firmware page has no ArduPilot option	QGC Daily build	Use QGC stable, or the uploader (Phase 3)

Symptom	Cause	Fix
`Permission denied` on `/dev/ttyACM0` or `/dev/ttyUSB0`	dialout group not active yet	Log out and back in
Uploader can't see the board	QGC or ModemManager holds the port	Close QGC; `sudo apt remove -y modemmanager`
"PreArm: check proximity sensor"	No lidar data	TX/RX, `SERIAL2_PROTOCOL` = 11, `SERIAL2_BAUD` = 460, 5 V at the C1
Pi stuck at "Waiting for heartbeat"	Link not set up	TX/RX, ground, `SERIAL5_PROTOCOL` / `SERIAL5_BAUD` = 2/921, `dtparam=uart0=on`
MAVProxy can't open the radio in the field	QGC auto-connected to it first	Untick SiK Radio in QGC's AutoConnect; start MAVProxy before QGC
QGC shows no vehicle in the field	Router not running, or UDP autoconnect off	Start the 8.2 router; tick UDP in QGC's AutoConnect

Script and tron messages

Message	Cause	Fix
`no heartbeat from the drone` / `no ArduPilot heartbeat`	Nothing on that UDP port	Check the MAVProxy (or SITL) command has the `--out` for that port
`not in GUIDED, refusing` / `need GUIDED`	Working as designed	Flip CH6 high (SITL: `mode guided`)
`rejected path planning off`	`GUID_OPTIONS` not 64 or `OA_TYPE` not 1	Set both, reboot
`rejected battery low (12.60 V)` in SITL	SITL simulates 3S by default	`param set SIM_BATT_VOLTAGE 16.8`
`rejected GPS not good enough`	Under 10 satellites or no 3D fix	Wait in the open; check the Pi isn't next to the GPS
`rejected too low`	Below 2 m	Climb in Loiter first

Message	Cause	Fix
`arming refused` from goto.py	A pre-arm check failed	Read the `FC:` lines above it, or QGC's messages
`stopped: pilot took over` / `override`	Mode left GUIDED	Working as designed
Pi camera import error after pip	pip's OpenCV shadows apt's	`pip uninstall opencv-python opencv-python-headless` in `~/tron-venv`
`pytest` can't find tron	Wrong venv	`apenv` (laptop) or `source ~/tron-venv/bin/activate` (Pi)

Sources

ArduPilot documentation:

- Holybro Pixhawk 6C: UART mapping, battery values, firmware folder
- Guided mode: `GUID_OPTIONS` bit 6 enables path planning in GUIDED
- Object avoidance with BendyRuler
- Simple object avoidance: Loiter and AltHold only
- RPLidar A1/A2/C1/S1 setup
- Communicating with Raspberry Pi via MAVLink
- Setting up the build environment and SITL on Linux
- Simulated peripherals: simulated 360° lidar and posts
- ArduPilot Methodic Configurator: starting tune values on Linux
- Copter stable firmware
- Optional Mission Planner extras: loading firmware, advanced tools (proximity view)

Other sources:

- Copter-4.7.2 release announcement (ArduPilot forum)
- Pixhawk6C board definition: board ID 56
- Pixhawk 6C port pinouts (Holybro)
- Pixhawk 6C power limits (PX4 docs)
- RPLIDAR C1 specifications (Waveshare)